

UNIT-1

Chapter-1

Introduction to data structures

1.1 Introduction

Introduces the subject of data structures and presents an overview of the content. Basic terminology and concepts will be defined and relevant examples provided. An overview of data organization and certain data structures will be covered along with different operations which are applied to these data structures.

1.2 Basic terminology; Elementary Data Organization

Data are simply *values* or *sets of values*. A **data item** refers to a *single unit of values*. Data items that are divided into subitems are called **group items**. That are not are called **elementary items**. For example, an employee's name may be divided into three subitems-first name, middle initial and last name-but the social security number would normally be treated as a single item. Collections of data are frequently organized into a hierarchy of **fields, records** and **files**. An **entity** is something that has certain attributes or properties which may be assigned values. The values themselves may be either numeric or non-numeric. For example, the following are possible attributes and their corresponding values for an entity, an employee of a given organization:

Attributes:	Name	Age	Sex	Social Security Number
Values:	Rohan	34	M	134-24-5533

Entities with similar attributes (e.g., all the employees in an organization) form an **entity set**. Each attribute of an entity set has a range of values, the set of all possible values that could be assigned to the particular attribute. The term "information" is sometimes used for data with given attributes, or, in other words, meaningful or processed data. The way that data are organized into the hierarchy of fields, records and files reflects the relationship between attributes, entities and entity sets. That is, a **field** is a single elementary unit of information representing an **attribute** of an entity, a **record** is the collection of field values of a given **entity** and a **file** is the collection of records of the entities in a given **entity set**. Each record in a file may contain many field items, but the value in a certain field may uniquely determine the record in the file. Such a field K is called a primary key, and the values $k_1, k_2, \dots$ in such a field are called **keys or key values**.

Example 1.1

- (a) Suppose an automobile dealership maintains an inventory file where each record contains the following data:

Serial Number, Type, Year, Price, Accessories

The Serial Number field can serve as a primary key for the file, since each automobile has a unique serial number.

- (b) Suppose an organization maintains a membership file where each record contains the following data:

Name, Address, Telephone Number, Dues Owed

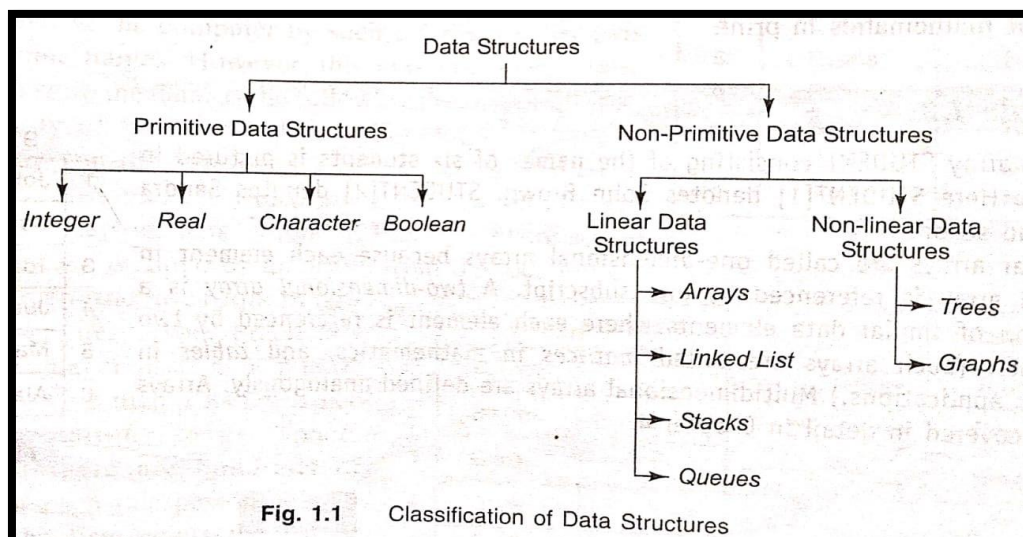
Although there are four data items, Name and Address may be group items. Here the Name field is a primary key. Note that the Address and Telephone Number fields may not serve as primary keys, since some members may belong to the same family and have the same address and telephone number.

Record also be classified according to length. A file can have **fixed-length records** or **variable-length records**. In **fixed-length records**, all the records contain the same data items with the same amount of space assigned to each data item. In variable-length records, file records may contain different lengths. For example, student records usually have variable lengths, since different students takes different number of courses. Usually, variable-length records have a minimum and maximum length.

1.3 Data Structures

The logical or mathematical model of a particular organisation of data is called a **data structure**. Data structure is the structural representation of logical relationship between data elements. This means that a data structure organizes data items based on the relationship between the data elements. The study of data structure helps you to understand how data is organized and how data flow is managed to increase efficiency of any process or program.

Classification of Data Structure



A data structure provides a structured set of variables that are associated with each other in different ways. It forms a basis of programming tool that represents the relationship between data elements and helps programmers to process the data easily.

Data structures are generally classified into *primitive* and *non-primitive* data structures. Basic data types such as integer, real, character and Boolean are known as primitive data structures. These data types consist of characters that cannot be divided, and hence they are also called *simple data types*.

The simplest example of non-primitive data structure is the processing of complex numbers. Very few computers are capable of doing arithmetic on complex numbers. Linked-lists, stacks, queues, trees and graphs are examples of non-primitive data structures. Figure 1. shows the classification of data structures.

Based on the structure and arrangement of data, non-primitive data structures are further classified into *linear* and *non-linear*. A data structure is said to be *linear* if its elements form a sequence or a linear list. In linear data structures, the data is arranged in a linear fashion although the way they are stored in memory need not be sequential. Arrays, linked lists, stacks and queues are examples of linear data structures.

Conversely, a data structure is said to be *non-linear* if the data is not arranged in sequence. The insertion and deletion of data is therefore not possible in a linear fashion. Trees and graphs are examples of non-linear data structures.

Arrays

An array is a linear data structure and it is a sequence collection of elements of same data type, stored at contiguous memory locations. Array, in general, refers to an orderly arrangement of data elements. Array is a type of data structure that stores data elements in adjacent locations. Array is considered as linear data structure that stores elements of same data types. Hence, it is also called as a linear homogenous data structure. The idea is to store multiple items of the same type together in one place. It allows the processing of a large amount of data in a relatively short period. The first element of the array is indexed by a subscript of 0. There are different operations possible in an array, like Searching, Sorting, Inserting, Traversing, Reversing, and Deleting.

By linear array, we mean list of a finite number n of similar data elements referenced respectively by a set of consecutive numbers, usually $1, 2, 3, \dots, n$. If we choose the name A for the array, then the elements of A are denoted by *subscript notation* $a_1, a_2, a_3, \dots, a_n$. or by the

Example 1.2

A linear array STUDENT consisting of the names of six students is pictured in Fig. 1.2. Here STUDENT[1] denotes John Brown, STUDENT[2] denotes Sandra Gold, and so on.

Linear arrays are called one-dimensional arrays because each element in such an array is referenced by one subscript. A *two-dimensional array* is a collection of similar data elements where each element is referenced by two subscripts. (Such arrays are called *matrices* in mathematics, and *tables* in business applications.) Multidimensional arrays are defined analogously. Arrays will be covered in detail in Chapter 4.

STUDENT	
1	John Brown
2	Sandra Gold
3	Tom Jones
4	June Kelly
5	Mary Reed
6	Alan Smith

Fig. 1.2

parenthesis notation $A(1), A(2), A(3), \dots, A(N)$ or by bracket notation $A[1], A[2], A[3], \dots, A[N]$. The number K in $A[K]$ is called a *subscript* and $A[K]$ is called as *Subscripted Variable*.

Linked List

A linked list is a data structure in which each data element contains a *pointer* or *link* and *data* or *info*. The *information field* contains the actual data. The *pointer field* contains address of the subsequent nodes in the list. The address of the first node a special name called *HEAD*. Also, the last node in the linked list can be identified because its next portion points to *NULL*. Through linked list, insertion and deletion of the data element is possible at all places of a linear list. Also in linked list, it is not necessary to have the data elements stored in consecutive locations. It allocates space for each data item in its own block of memory. Thus, a linked list is considered as a chain of data elements or records called nodes.

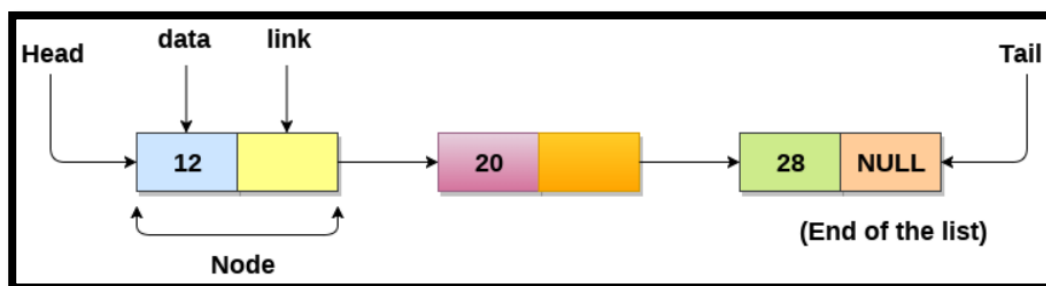


Fig 1.2 Linked List

Stack

Stack is a linear data structure that follows a particular order in which the operations are performed. The order may be LIFO (Last In First Out) or FILO (First In Last Out). LIFO implies that the element that is inserted last, comes out first and FILO implies that the element that is inserted first, comes out last. In stack insertion and deletion of elements are done at only one end, which is known as the top of the stack.

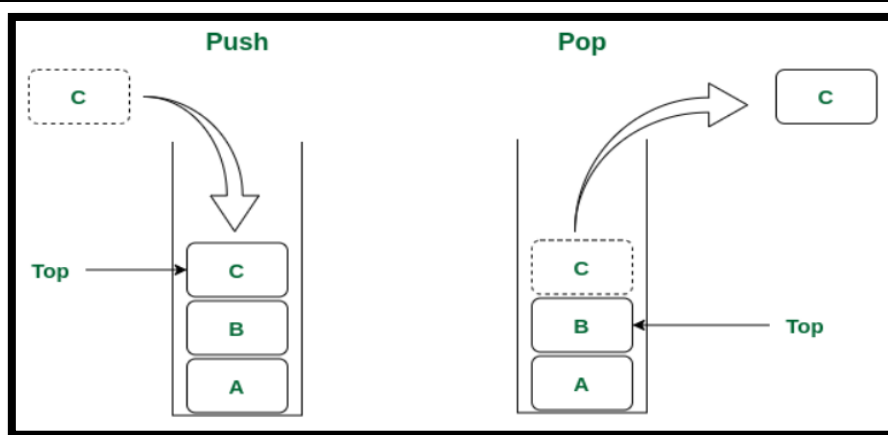


Fig. 1.3 Stack

Queue

Queue is a linear data structure in which the first element is inserted from one end called the REAR (also called tail), and the removal of existing element takes place from the other end called as FRONT (also called head). This makes queue as FIFO (First in First Out) data structure, which means that element inserted first will be removed first. Which is exactly how queue system works in real world. If you go to a ticket counter to buy movie tickets, and are first in the queue, then you will be the first one to get the tickets. Same is the case with Queue data structure. Data inserted first, will leave the queue first. The process to add an element into queue is called Enqueue and the process of removal of an element from queue is called Dequeue. Queues can be implemented by using either arrays or linked lists.



Fig. 1.4 Queue

Trees

A *tree* is a *non-linear data structure* in which data is organized in branches. A tree data structure is defined as a *collection of objects or entities known as nodes* that are linked together to represent or simulate hierarchy. It does not store the elements in a sequential manner. The data elements in tree are arranged in a sorted order. It imposes a hierarchical structure on the data elements. The data structure which reflects this relationship is called a *rooted tree graph* or, simply, a *tree*. In the Tree data structure, the topmost node is known as a root node. Each node contains some data, and data can be of any type. In the below tree structure, the node

contains the name of the employee, so the type of data would be a string. Each node contains some data and the link or reference of other nodes that can be called *children*.

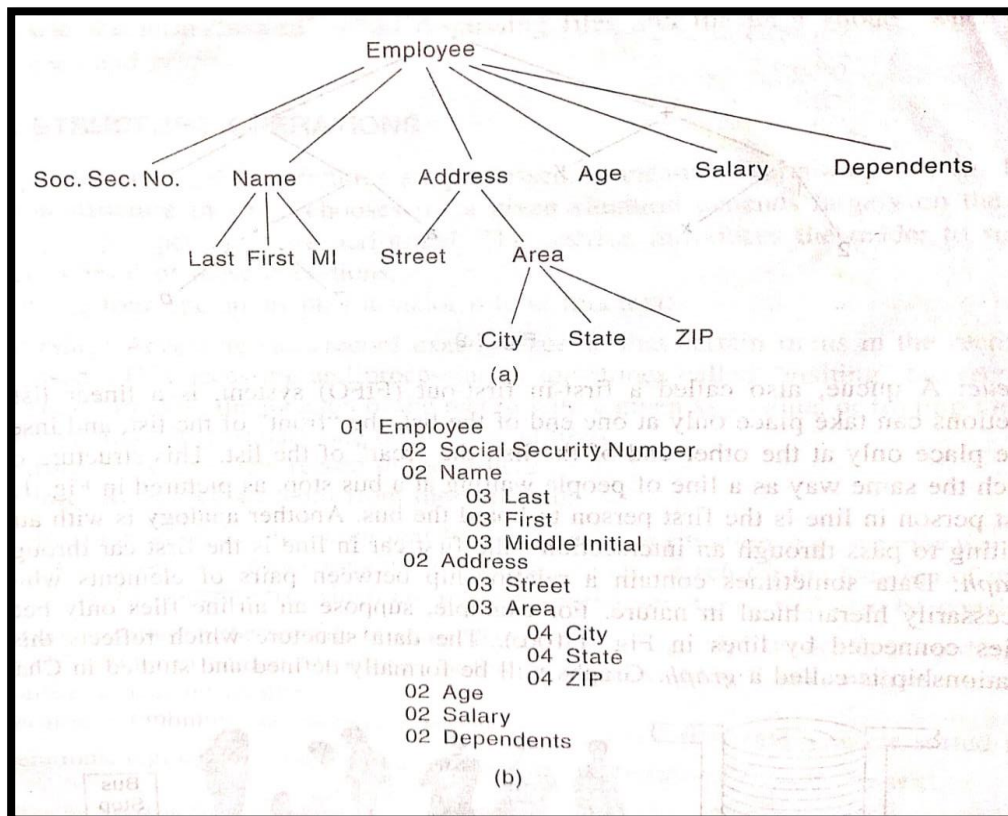


Fig. 1.5 Tree

Graphs

A graph is a non-linear data structure, can be defined as group of vertices and edges that are used to connect these vertices. A graph can be seen as a cyclic tree, where the vertices (Nodes) maintain any complex relationship among them instead of having parent child relationship. A graph G can be defined as an ordered set $G(V, E)$ where $V(G)$ represents the set of vertices and $E(G)$ represents the set of edges which are used to connect these vertices.

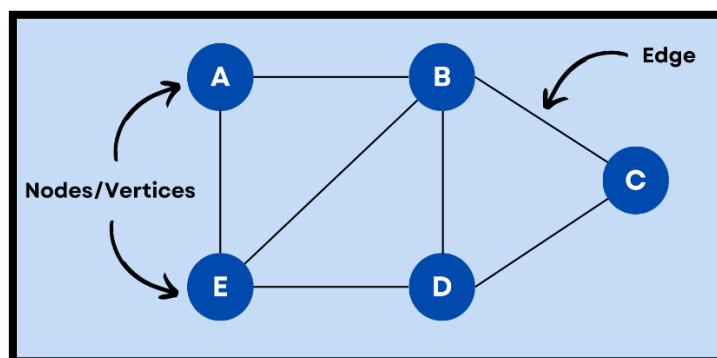


Fig. 1.6 Graph

1.4 Operations on Data Structure

The data appearing in our data structure are processed by means of certain operation. In fact, the Particular data structure that one chooses for a given situation depends largely on the frequency. With which specific operation are performed. The following four operation play a major role

1. **Traversing:** accessing each record exactly Once so that certain item in the record maybe processed.
2. **Searching:** Finding the location of the record with a given key value. or finding the location of all records which satisfy one or more condition.
3. **Inserting:** adding a new record to the structure.
4. **Deleting:** removing a record from the structure.

Sometimes two or more of operations may be used in a given situation; e. g., we may want to delete the record with a given k which may mean we first need to search for the location of the record.

The following two Operations, which are used in Special Situation, will also be considered:

5. **Sorting:** Arranging the records in some logical order. (e.g., alphabetically according to some NAME key, Or in numerical order according to some NUMBER key, Such as Social Security number or account number)
6. **Merging:** combining the records in two different sorted files into a single sorted file other operation, e.g., Coping and concatenation.

UNIT-1

Chapter-2 Preliminaries

2.1 Introduction

In developing software application, if the problem is complex, then it is divided into the sub-modules. After dividing individual modules, they are combined to form a complete software application. The main module gives the general description of the algorithm. The sub-program contains more detailed information than the main module.

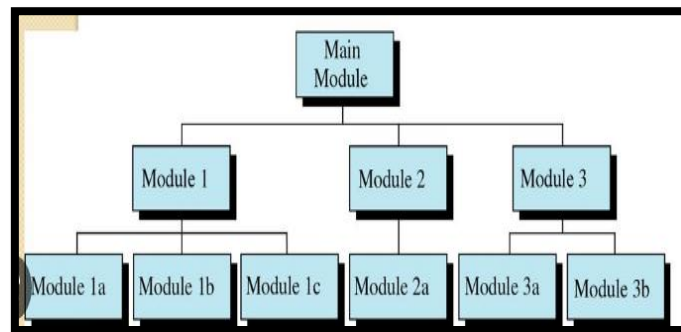


Fig 2.1 Hierarchy of Modules

2.2 Algorithmic Notations

An algorithm is a finite step-by-step list of well-defined instructions for solving a particular problem. It gives a crisp solution. It is not the complete program or code; it is just a solution (logic) of a problem, which can be represented either as an informal description using

Example 2.4

An array DATA of numerical values is in memory. We want to find the location LOC and the value MAX of the largest element of DATA. Given no other information about DATA, one way to solve the problem is as follows:

Initially begin with $LOC = 1$ and $MAX = DATA[1]$. Then compare MAX with each successive element $DATA[K]$ of DATA. If $DATA[K]$ exceeds MAX, then update LOC and MAX so that $LOC = K$ and $MAX = DATA[K]$. The final values appearing in LOC and MAX give the location and value of the largest element of DATA.

A formal presentation of this algorithm, whose flow chart appears in Fig. 2.2, follows.

Algorithm 2.1: (Largest Element in Array) A nonempty array DATA with N numerical values is given. This algorithm finds the location LOC and the value MAX of the largest element of DATA. The variable K is used as a counter.

- Step 1. [Initialize.] Set $K := 1$, $LOC := 1$ and $MAX := DATA[1]$.
- Step 2. [Increment counter.] Set $K := K + 1$.
- Step 3. [Test counter.] If $K > N$, then:
Write: LOC, MAX, and Exit.
- Step 4. [Compare and update.] If $MAX < DATA[K]$, then:
Set $LOC := K$ and $MAX := DATA[K]$.
- Step 5. [Repeat loop.] Go to Step 2.

a Flowchart or Pseudocode. Algorithms are generally created independent of underlying languages, i.e., an algorithm can be implemented in more than one programming language. It is easy to understand. In Algorithm the problem is broken down into smaller pieces or steps hence, it is easier for the programmer to convert it into an actual program.

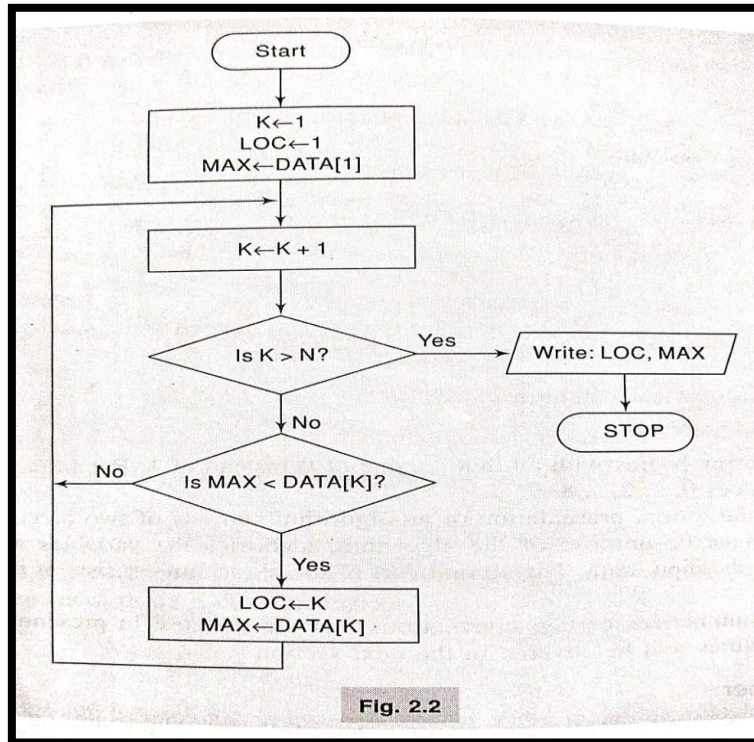


Fig 2.2 Flowchart

C implementation of the algorithm 2.1

```

#include<stdio.h>
#include<conio.h>
void main()
{
    int data[10]={22, 65, 1, 99, 32, 17, 74, 49, 33, 21}
    int n, loc, max, k;
    n=10;
    k=0;
    loc=0;
    max=data[0];
    clrscr();
    loop:
        k=k+1;
        if(k==n)
        {
            printf("Location=%d  Maximum=%d",loc,max);
            getch();
            exit();
        }
        if(max>data[k])
        {
            loc=k;
            max=data[k];
        }
        goto loop;
}
OUTPUT: Location=3  Maximum=99
    
```

The format for formal presentation of an algorithm consists of two parts:

- The first part is a paragraph which tells the **purpose of the algorithm, identifies the variable which occur in the algorithm and list of input data.**
- The second part of the algorithm consists of the **list of steps that is to be executed.**

The notations to represent the algorithm are as follows

1. Identifying Number:

Each algorithm is assigned an identifying number as follows:

- a) Algorithm 4.3 refers to the third algorithm in Chapter 4.
- b) Algorithm P5.3 refer to the algorithm in Solved Problem 5.3 in Chapter 5. Here 'P' represents the algorithm appears in a problem.

2. Steps, Control, Exit

The steps of the algorithm are executed one after the other, beginning with the step 1, unless indicated otherwise. Control may be transferred to Step n of the algorithm by the statement "Go to Step n". For example: Step 5 transfers control back to Step 2 in previous algorithm. If several statements appear in the same step, they are executed from left to right. Example:

Set K:=1, LOC:=1 and MAX:= DATA[1]

The algorithm is completed when the "Exit" statement is encountered.

3. Comments

Each step may contain a comment in brackets which indicates the main purpose of the step. The comment will usually appear in the beginning or the end of the step. Example:

Step 1: [Initialize] Set K:=1, LOC:=1 and MAX:= DATA[1] or

Step 1: Set K:=1, LOC:=1 and MAX:= DATA[1] [Initialize]

4. Variable Names

Variable Names will use capital letters, as in MAX and DATA. Single-letter names of variables used as counters or subscripts will also be capitalized in algorithms. E. g: K and N.

5. Assignment Statements

Assignment statements will use the dots-equal notation := Example: MAX := DATA[1] will assigns the value in DATA[1] to MAX.

6. Input and Output

Data may be input and assigned to variable by means of a Read statement with the Form: **Read: Variable Names.** Similarly, messages, placed in quotation marks, and data in variables may be output by means of a Write or print statement with the Form: **Write: Messages and/or Variable names**

7. Procedures

The term procedure will be used for an independent algorithmic module which solves a particular problem. The use of the word “procedure” or “module” rather than “algorithm” for a given problem is simply a matter of taste. The word “algorithm” will be reserved for the solution of general problems. The term “procedure” will also be used to describe a certain type of sub-algorithm.

2.3 Control Structures

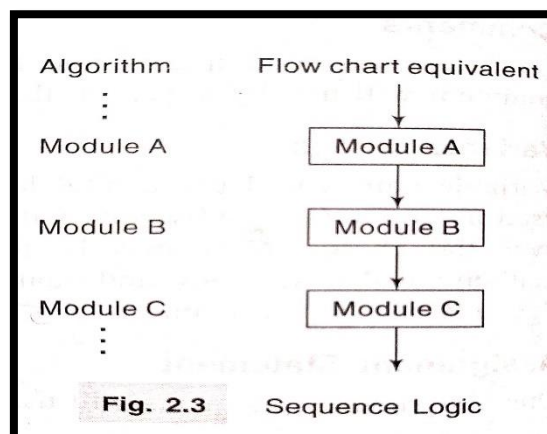
Algorithms and their equivalent computer programs are more easily understood if they mainly use self-contained modules and three types of logic, or flow of control, called

1. Sequence logic, or sequential flow
2. Selection logic, or conditional flow
3. Iteration logic, or repetitive flow.

These three types of logic are discussed below, and in each case we show the equivalent flowchart.

1. Sequence logic, or sequential flow

Unless instructions are given to the contrary, the modules are executed in the obvious sequence. The sequence may be presented explicitly, by means of numbered steps, or implicitly, by the order in which the modules are written. (See Fig. 2.3.) Most processing, even of complex problems, will generally follow this elementary flow pattern.



2. Selection Logic (Conditional Flow)

Selection logic employs a number of conditions which lead to a selection of one out of several alternative modules. The structures which implement this logic are called conditional structures or If structures. For clarity, we will frequently indicate the end of such a structure by the statement.

[End of If structure.]

These conditional structures fall into three types

a) Single Alternative.

This structure has the form

If condition, then:

[Module A]

[End of If structure.]

The logic of this structure is pictured in Fig. 2.4(a). If the condition holds, then Module A, which may consist of one or more statements, is executed; Otherwise, Module A is skipped and control transfers to the next step of the algorithm.

b) Double Alternative.

This structure has the form

If condition, then:

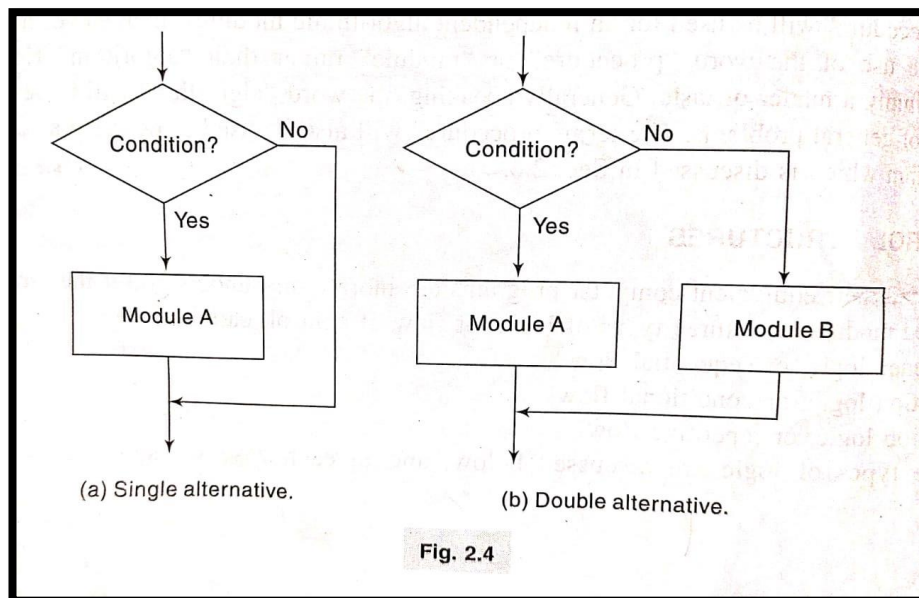
[Module A]

Else:

[Module B]

[End of If structure)

The logic of this structure is pictured in Fig. 2.4(b). As indicated by the flow chart, if the condition holds, then Module A is executed; otherwise, Module B is executed.



c) Multiple Alternatives.

This structure has the form

If condition(1), then:

[Module A,]

Else if condition(2), then:

[Module A₂]

Else if condition(M), then:

[Module A_M]

Else:

[Module B]

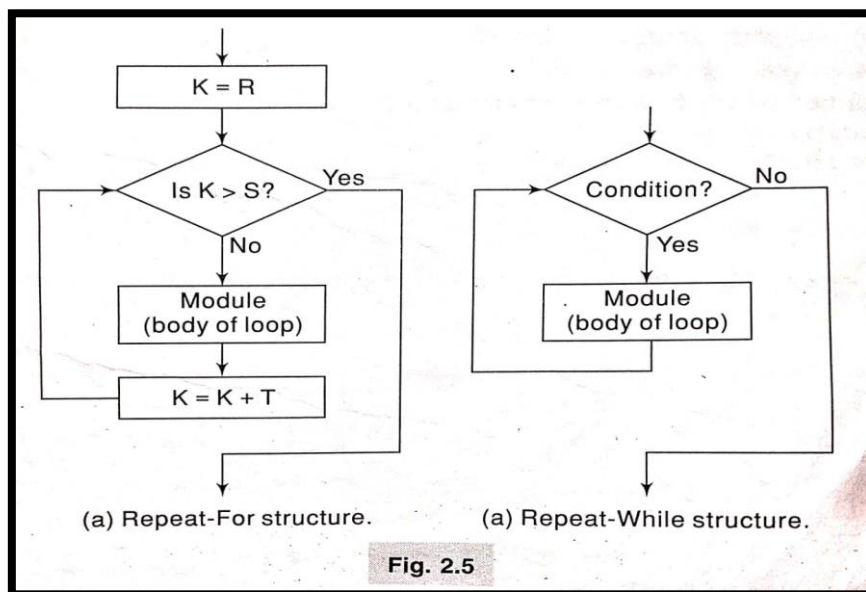
[End of If structure.]

The logic of this structure allows only one of the modules to be executed. Specifically, either the module which follows the first condition which holds is executed, or the module which follows the final Else statement is executed. In practice, there will rarely be more than three alternatives.

3. Iteration logic, or repetitive flow.

The Iteration Logic involves two types of loops. That is repeat-for loop and repeat-while loop. Each type begins with a Repeat statement and is followed by body of the loop. The end of the structure by the statement:

[End of loop.]



The **repeat-for loop** uses an index variable, such as K, to control the loop. The loop will usually have the form:

Repeat for K = R to S by T:

[Module]

[End of loop.]

The logic of this structure is pictured in Fig. 2.5(a). Here R is called the initial value, S the end value or test value, and T the increment. Observe that the body of the loop is executed first with $K = R$, then with $K = R + T$, then with $K = R + 2T$, and so on. (assume $R=0$, $S=5$, $T=1$ then $K=0+1=1$, next step $K=0+2*1$ and so on.). The cycling ends when $K > S$. The

flow chart in the next slide assumes that the increment T is positive; if T is negative, so that K decreases in value, then the cycling ends when $K < S$.

The **repeat-while loop** uses a condition to control the loop. The loop will usually have the form

Repeat while condition:

[Module]

[End of loop.]

The logic of this structure is pictured in Fig. 2.5(b). Observe that the cycling continues until the condition is false.

Unit-1

Chapter-3

Recursion**3.1 Definition**

The process in which a function calls itself directly or indirectly is called **recursion** and the corresponding function is called a **recursive function**. A **recursive function solves a particular problem by calling a copy of itself and solving smaller subproblems of the original problems**. Suppose P is a procedure containing either a Call statement to itself or a Call statement to a second procedure that may eventually result in a Call statement back to the original procedure P. Then P is called a **recursive procedure**. So that the program will not continue to run indefinitely, a recursive procedure must have the following two properties:

1. There must be certain criteria, called base criteria, for which the procedure does not call itself.
2. Each time the procedure does call itself (directly or indirectly), it must be closer to the base criteria.

A recursive procedure with these two properties is said to be well-defined.

Similarly, a function is said to be recursively defined if the function definition refers to itself.

Again, in order for the definition not to be circular, it must have the following two properties:

1. There must be certain arguments, called base values, for which the function does not refer to itself.
2. Each time the function does refer to itself, the argument of the function must be closer to a base value.

A recursive function with these two properties is also said to be well-defined.

Factorial Function

The product of the positive integers from 1 to n , inclusive, is called " n factorial" and is usually denoted by $n!$:

$$n! = 1 \cdot 2 \cdot 3 \cdot \dots \cdot (n-2) \cdot (n-1) \cdot n$$

It is also convenient to define $0! = 1$, so that the function is defined for all non-negative integers. Thus, we have

$$0! = 1 \quad 1! = 1 \quad 2! = 1 \cdot 2 = 2 \quad 3! = 1 \cdot 2 \cdot 3 = 6 \quad 4! = 1 \cdot 2 \cdot 3 \cdot 4 = 24$$

$$5! = 1 \cdot 2 \cdot 3 \cdot 4 \cdot 5 = 120 \quad 6! = 1 \cdot 2 \cdot 3 \cdot 4 \cdot 5 \cdot 6 = 720$$

and so on. Observe that

$$5! = 1 \cdot 2 \cdot 3 \cdot 4 \cdot 5 = 120 \quad 6! = 1 \cdot 2 \cdot 3 \cdot 4 \cdot 5 \cdot 6 = 720$$

This is true for every positive integer n ; that is,

$$n! = n \cdot (n-1)!$$

Accordingly, the factorial function may also be defined as follows:

Definition 6.1: (Factorial Function)

- (a) If $n = 0$, then $n! = 1$.
- (b) If $n > 0$, then $n! = n \cdot (n - 1)!$

Observe that this definition of $n!$ is recursive, since it refers to itself when it uses $(n-1)!$

However,

- a) The value of $n!$ is explicitly given when $n = 0$ (thus 0 is the base value); and
- b) The value of $n!$ for arbitrary n is defined in terms of a smaller value of n which is closer to the base value 0. Accordingly, the definition is not circular, or in other words, the procedure is well- defined.

Let us calculate $4!$ using the recursive definition. This calculation requires the following nine steps

```

1. 4! = 4 · 3!
2.   3! = 3 · 2!
3.    2! = 2 · 1!
4.     1! = 1 · 0!
5.      0! = 1
6.     1! = 1 · 1 = 1
7.    2! = 2 · 1 = 2
8.   3! = 3 · 2 = 6
9.  4! = 4 · 6 = 24
  
```

That is:

Step 1. This defines $4!$ in terms of $3!$, so we must postpone evaluating $4!$ until we evaluate $3!$

This postponement is indicated by indenting the next step.

Step 2. Here $3!$ is defined in terms of $2!$, so we must postpone evaluating $3!$ until we evaluate $2!$

Step 3. This defines $2!$ in terms of $1!$

Step 4. This defines $1!$ in terms of $0!$

Step 5. This step can explicitly evaluate $0!$, since 0 is the base value of the recursive definition.

Steps 6 to 9. We backtrack, using $0!$ to find $1!$, using $1!$ to find $2!$, using $2!$ to find $3!$, and finally using $3!$ to find $4!$ This backtracking is indicated by the "reverse" indentation.

Observe that we backtrack in the reverse order of the original postponed evaluations. Recall that this type of postponed processing lends itself to the use of stacks.

The following are the two procedures that each calculate n Factorial

Procedure 6.9A: FACTORIAL(FACT, N)

This procedure calculates $N!$ and returns the value in the variable FACT.

1. If $N = 0$, then: Set $FACT := 1$, and Return.
2. Set $FACT := 1$. [Initializes FACT for loop.]
3. Repeat for $K = 1$ to N .
 Set $FACT := K * FACT$.
 [End of loop.]
4. Return.

Procedure 6.9B: FACTORIAL(FACT, N)

This procedure calculates $N!$ and returns the value in the variable FACT.

1. If $N = 0$, then: Set $FACT := 1$, and Return.
2. Call $FACTORIAL(FACT, N - 1)$.
3. Set $FACT := N * FACT$.
4. Return.

Fibonacci Sequence

The Fibonacci sequence (usually denoted by F_0, F_1, F_2) is as follows:

0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55,

That is, $F_0 = 0$ and $F_1 = 1$ and each succeeding term is the sum of the two preceding terms. For example, the next two terms of the sequence are

$$34 + 55 = 89 \quad \text{and} \quad 55 + 89 = 144$$

The Formal Definition of this function as follows:

Definition 6.2: (Fibonacci Sequence)

- a) If $n = 0$ or $n = 1$, then $F_n = n$.
- b) If $n > 1$, then $F_n = F_{n-2} + F_{n-1}$

This is another example of a recursive definition, since the definition refers to itself when it uses F_{n-2} and F_{n-1} . Here,

- (a) the base values are 0 and 1, and
- (b) the value of F_n , is defined in terms of smaller values of n which are closer to the base values.

Accordingly, this function is well-defined.

A procedure for finding the n^{th} term F_n of the Fibonacci Sequence as follows:

Procedure 6.10: FIBONACCI(FIB, N)

This procedure calculates F_N and returns the value in the first parameter FIB.

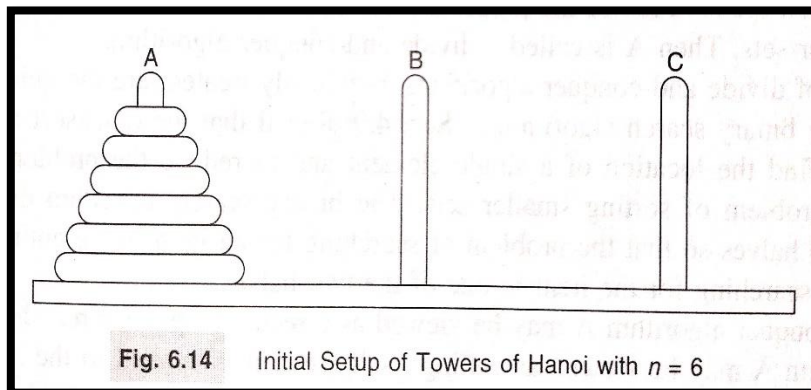
1. If $N = 0$ or $N = 1$, then: Set $FIB := N$, and Return.
2. Call $FIBONACCI(FIBA, N - 2)$.
3. Call $FIBONACCI(FIBB, N - 1)$.
4. Set $FIB := FIBA + FIBB$.
5. Return.

This is another example of a recursive procedure, since the procedure contains a call to itself. In fact, this procedure contains two calls to itself.

Towers of Hanoi

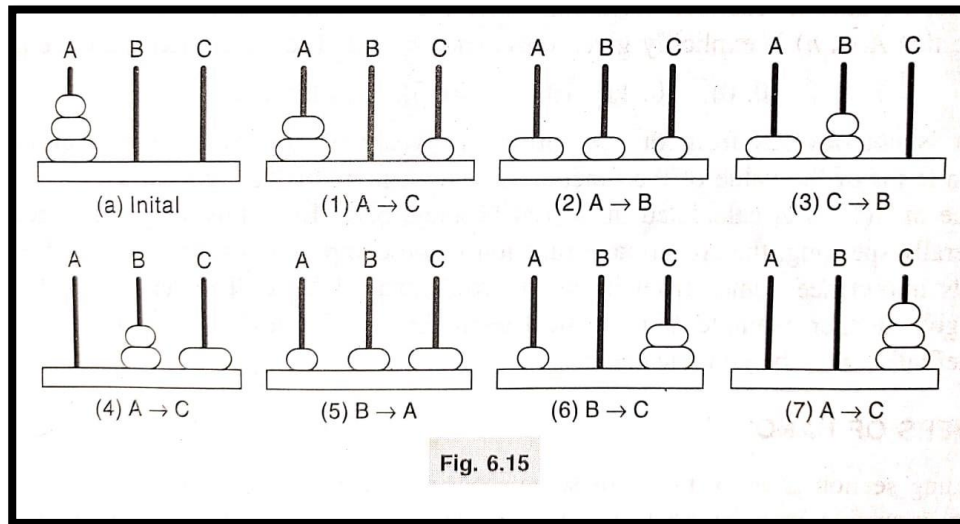
This section shows how recursion may be used as a tool in developing an algorithm to solve a particular problem. The problem is known as the **Towers of Hanoi problem**. Suppose three pegs, labeled A, B and C, are given, and suppose on peg A there are placed a finite number n of disks with decreasing size. This is pictured in Fig. 6.14 for the case $n = 6$. The object of the game is to move the disks from peg A to peg C using peg B as an auxiliary. The rules of the game are as follows:

- a) Only one disk may be moved at a time. Specifically, only the top disk on any peg may be moved to any other peg.
- b) At no time can a larger disk be placed on a smaller disk.



Sometimes we will write $X \rightarrow Y$ to denote the instruction "Move top disk from peg X to peg Y," where X and Y may be any of the three pegs. The solution to Towers of Hanoi problem for $n = 3$ appears in Fig. 6.15 Observe that it consists of the following seven moves:

- $n = 3$:
- Move top disk from peg A to peg C.
 - Move top disk from peg A to peg B.
 - Move top disk from peg C to peg B.
 - Move top disk from peg A to peg C.
 - Move top disk from peg B to peg A.
 - Move top disk from peg B to peg C.
 - Move top disk from peg A to peg C.



In other words, $n=3$: $A \rightarrow C$, $A \rightarrow B$, $C \rightarrow B$, $A \rightarrow C$, $B \rightarrow A$, $B \rightarrow C$, $A \rightarrow C$

For completeness, we give the solution to the Towers of Hanoi problem for $n = 1$ and $n = 2$:

$n = 1$: $A \rightarrow C$

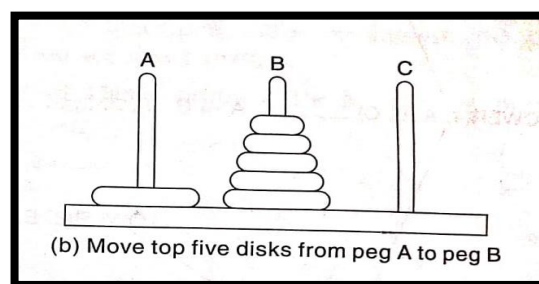
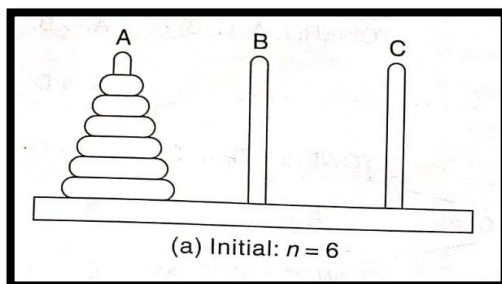
$n = 2$: $A \rightarrow B$, $A \rightarrow C$, $B \rightarrow C$

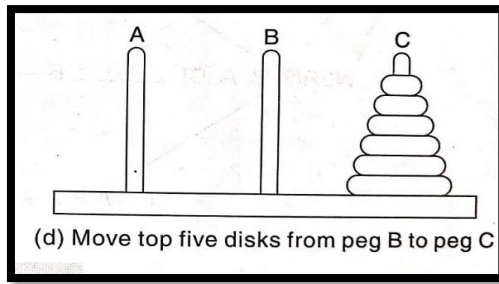
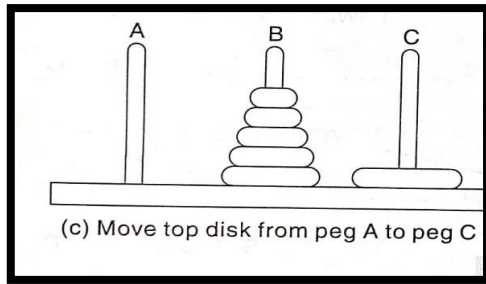
Note that $n = 1$ uses only one move and that $n = 2$ uses three moves.

Rather than finding a separate solution for each n , we use the technique of recursion to develop a general solution. First we observe that the solution to the Towers of Hanoi problem for $n > 1$ disks may be reduced to the following subproblems:

1. Move the top $n - 1$ disks from peg A to peg B.
2. Move the top disk from peg A to peg C: $A \rightarrow C$.
3. Move the top $n - 1$ disks from peg B to peg C.

This reduction is illustrated in Fig. 6.16 for $n = 6$. That is, first we move the top five disks from peg A to peg B, then we move the large disk from peg A to peg C, and then we move the top five disks from peg B to peg C.





Let us now introduce the general notation

$TOWER(N, BEG, AUX, END)$

to denote a procedure which moves the top n disks from the initial peg BEG to the final peg END using the peg AUX as an auxiliary. When $n = 1$, we have the following obvious solution:

$TOWER(1, BEG, AUX, END)$ consists of the single instruction $BEG \rightarrow END$

Furthermore, as discussed above, when $n > 1$, the solution may be reduced to the solution of the following three subproblems:

1. $TOWER(N-1, BEG, END, AUX)$
2. $TOWER(1, BEG, AUX, END)$ or $BEG \rightarrow END$
3. $TOWER(N-1, AUX, BEG, END)$

Observe that each of these three subproblems may be solved directly or is essentially the same as the original problem using fewer disks. Accordingly, this reduction process does yield a recursive solution to the Towers of Hanoi Problem.

Procedure 6.11: $TOWER(N, BEG, AUX, END)$

This procedure gives a recursive solution to the Towers of Hanoi problem for N disks.

1. If $N = 1$, then:
 - (a) Write: $BEG \rightarrow END$.
 - (b) Return.

[End of If structure.]
2. [Move $N - 1$ disks from peg BEG to peg AUX .]
Call $TOWER(N - 1, BEG, END, AUX)$.
3. Write: $BEG \rightarrow END$.
4. [Move $N - 1$ disks from peg AUX to peg END .]
Call $TOWER(N - 1, AUX, BEG, END)$.
5. Return.

/*Program to solve Tower of Hanoi using Recursion. */

```
#include<stdio.h>
#include<conio.h>
void Tower(int,char[],char[],char[]);
void main()
{
    int n;
    clrscr();
    printf("\nEnter the Number of Disks to be Transferred:");
    scanf("%d",&n);
    if(n<1)
    {
        printf("\nIncorrect Value");
        exit();
    }
    else
    {
        printf("\nThe following Moves are required for N=%d\n\n",n);
        Tower(n,"Beg","Aux","End");
    }
    getch();
}
void Tower(int num,char A[5],char B[5],char C[5])
{
    if(num==1)
    {
        printf("%s --> %s \t",A,C);
        return;
    }
    Tower(num-1,A,C,B);
    printf(" %s --> %s \t",A,C);
    Tower(num-1,B,A,C);
    return;
}
```

OUTPUT:

Enter the number of disks to be transferred: 3

The following moves are required for N = 3

BEG -> END AUX -> BEG BEG-> AUX END -> AUX BEG -> END AUX -> END
BEG -> END

UNIT-1

Chapter – 4
Arrays

4.1 Introduction

Data structures are classified as either *linear* or *nonlinear*. A data structure is said to be *linear* if its elements form a sequence, or, in other words, a *linear list*. There are two basic ways of representing such linear structures in memory. One way is to have the linear relationship between the elements represented by means of sequential memory locations. These linear structures are called *arrays*. The other way is to have the linear relationship between the elements represented by means of pointers or links. These linear structures are called *linked lists*. Nonlinear structures such as *trees* and *graphs*. The operations one normally performs on any linear structure, whether it be an array or a linked list, include the following:

- a) *Traversal*. Processing each element in the list.
- b) *Search*. Finding the location of the element with a given value or the record with a given key.
- c) *Insertion*. Adding a new element to the list.
- d) *Deletion*. Removing an element from the list.
- e) *Sorting*. Arranging the elements in some type of order.
- f) *Merging*. Combining two lists into a single list.

Arrays are usually easy to traverse, search and sort, they are frequently used to store relatively permanent collections of data. On the other hand, if the size of the structure and the data in the structure are constantly changing, then the array may not be as useful structure as the linked list.

4.2 Linear Arrays

A *linear array* is a list of a finite number n of *homogeneous* data elements (i.e., data elements of the same type) such that:

- a) The elements of the array are referenced respectively by an *index set* consisting of n consecutive numbers.
- b) The elements of the array are stored respectively in successive memory locations.

The number n of elements is called the *length* or *size* of the array. In general, the length or the number of data elements of the array can be obtained from the index set by the formula

$$\text{Length} = \text{UB} - \text{LB} + 1$$

where UB is the largest index, called the *upper bound*, and LB is the smallest index, called the *lower bound*, of the array.

Note that length = UB when LB = 1.

The elements of an array A may be denoted by the subscript notation

$$A_1, A_2, A_3, \dots, A_n$$

or by the bracket notation (used in C)

$$A[1], A[2], A[3], \dots, A[N]$$

We will usually use the *subscript* notation or the *bracket* notation. The number K in A[K] is called a *subscript* or an *index* and A[K] is called a *subscripted variable*.

Example 4.1

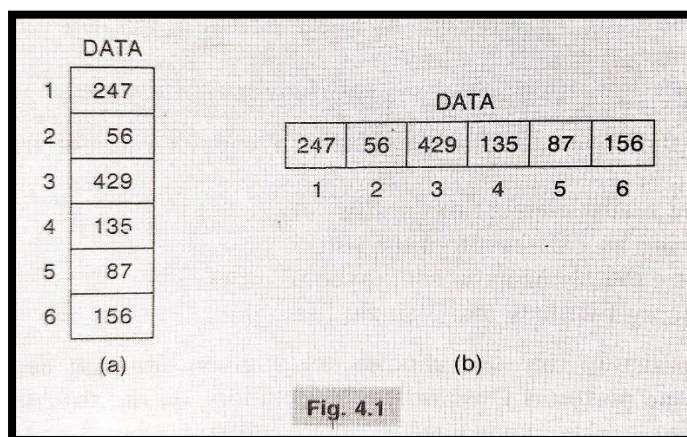
a) Let DATA be a 6-element linear array of integers such that

$$\text{DATA}[1]=247, \text{DATA}[2]=56, \text{DATA}[3]=429, \text{DATA}[4]=135, \text{DATA}[5]=87, \text{DATA}[6]=156$$

Sometimes we will denote such an array by simply writing

$$\text{DATA: } 247, 56, 429, 135, 87, 156$$

The array DATA is frequently pictured as in Fig. 4.1(a) or Fig. 4.1(b).



b) An automobile company uses an array AUTO to record the number of automobiles sold each year from 1932 through 1984.

Rather than beginning the index set with 1, it is more useful to begin the index set with 1932 so that

$$\text{AUTO}[K] = \text{number of automobiles sold in the year } K.$$

Then LB 1932 is the lower bound and UB = 1984 is the upper bound of AUTO.

$$\text{Length} = \text{UB} - \text{LB} + 1$$

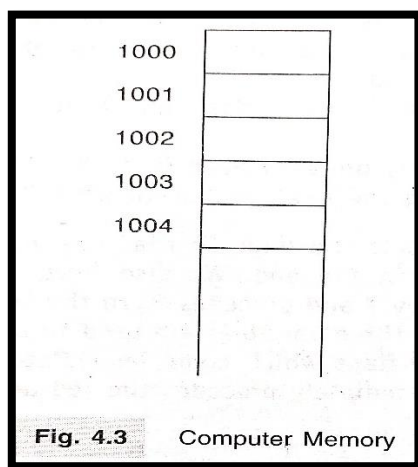
$$= 1984 - 1932 + 1$$

$$= 53$$

That is, AUTO contains 53 elements and its index set consists of all integers from 1932 through 1984.

4.3 Representation of Linear Arrays in Memory

Let LA be a linear array in the memory of the computer. Recall that the memory of the computer is simply a sequence of addressed locations as pictured in Fig. 4.3.



Let us use the notation

$LOC(LA[K])$ = address of the element $LA[K]$ of the array LA

The elements of LA are stored in successive memory cells. Accordingly, the computer does not need to keep track of the address of every element of LA, but needs to keep track only of the address of the first element of LA, denoted

Base (LA)

and called the **base address** of LA. Using this address $Base (LA)$, the computer calculates the address of any element of LA by the following formula:

$$LOC (LA[K]) = Base (LA) + w(K - \text{lower bound})$$

where w is the number of words per memory cell for the array LA. Observe that the time to calculate $LOC(LA[K])$ is essentially the same for any value of K. Furthermore, given any subscript K, one can locate and access the content of $LA[K]$ without scanning any other element of LA.

Example 4.4

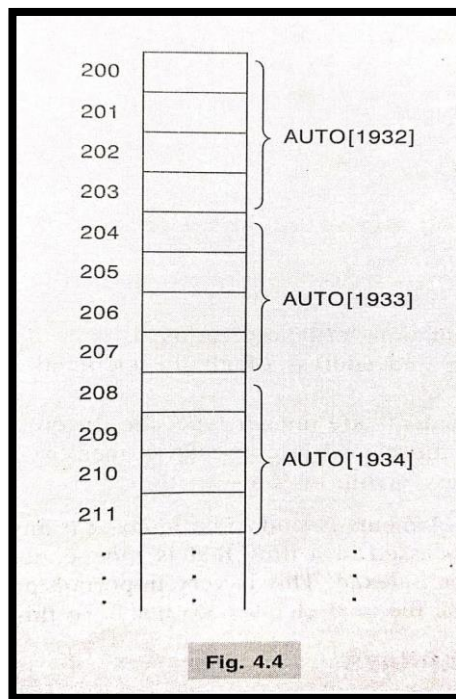
Consider the array AUTO in below example, which records the number of automobiles sold each year from 1932 through 1984. Suppose AUTO appears in memory as pictured in Fig. 4.4. That is, $Base (AUTO)=200$, and $w = 4$ words per memory cell for AUTO. Then,

$LOC (AUTO [1932]) = 200$, $LOC (AUTO [1933]) = 204$, $LOC (AUTO [1934]) = 208$, ...

The address of the array element for the year $K = 1965$ can be obtained by using Eq. (4.2):

$$\begin{aligned} LOC (AUTO [1965]) &= Base (AUTO) + w (1965 - \text{lower bound}) \\ &= 200 + 4(1965 - 1932) \\ &= 332 \end{aligned}$$

Again we emphasize that the contents of this element can be obtained without scanning any other element in an array AUTO



4.4 Traversing Linear Arrays

Let A be a collection of data elements stored in the memory of the computer. Suppose we want to print the contents of each element of A or suppose we want to count the number of elements of A with a given property. This can be accomplished by traversing A , that is, by accessing and processing (frequently called visiting) each element of A exactly once.

The following algorithm traverses a linear array LA . The simplicity of the algorithm comes from the fact that LA is a *linear structure*. Other linear structures, such as linked lists, can also be easily traversed. On the other hand, the traversal of nonlinear structures, such as trees and graphs, is considerably more complicated.

Algorithm 4.1: (Traversing a Linear Array) Here LA is a linear array with lower bound LB and upper bound UB . This algorithm traverses LA applying an operation $PROCESS$ to each element of LA .

1. [Initialize counter.] Set $K := LB$.
 2. Repeat Steps 3 and 4 while $K \leq UB$.
 3. [Visit element.] Apply $PROCESS$ to $LA[K]$.
 4. [Increase counter.] Set $K := K + 1$.
- [End of Step 2 loop.]
4. Exit.

4.5 Inserting and Deleting

Let A be a collection of data elements in the memory of the computer. "**Inserting**" refers to the operation of adding another element to the collection A , and "**deleting**" refers to the operation of removing one of the elements from A . This section discusses inserting and deleting when A is a linear array.

Inserting an element at the "end" of a linear array can be easily done provided the memory space allocated for the array is large enough to accommodate the additional element. On the other hand, suppose we need to insert an element in the middle of the array. Then, on the average, half of the elements must be moved downward to new locations to accommodate the new element and keep the order of the other elements.

Similarly, deleting an element at the "end" of an array presents no difficulties, but deleting an element somewhere in the middle of the array would require that each subsequent element be moved one location upward in order to "fill up" the array.

Remark: Since linear arrays are usually pictured extending downward, as in Fig. 4.1, the term "downward" refers to locations with larger subscripts, and the term "upward" refers to locations with smaller subscripts.

Example 4.6

Suppose TEST has been declared to be a 5-element array but data have been recorded only for TEST [1], TEST [2] and TEST [3]. If X is the value of the next test, then one simply assigns

$$\text{TEST}[4] = X$$

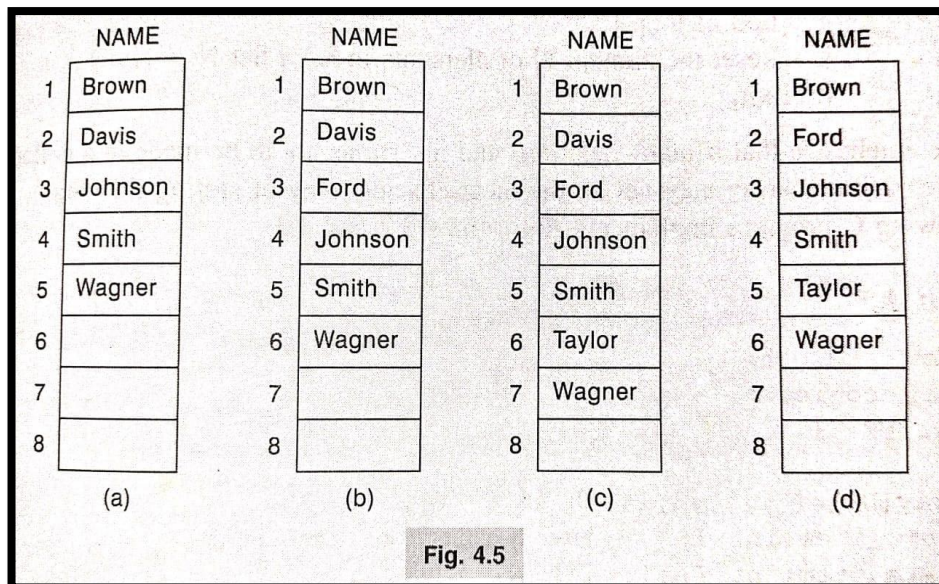
to add X to the list. Similarly, if Y is the value of the subsequent test, then we simply assign

$$\text{TEST}[5] = Y$$

to add Y to the list. Now, however, we cannot add any new test scores to the list.

Example 4.7

Suppose NAME is an 8-element linear array, and suppose five names are in the array, as in Fig. 4.5(a). Observe that the names are listed alphabetically, and suppose we want to keep the array names alphabetical at all times. Suppose Ford is added to the array. Then Johnson, Smith and Wagner must each be moved downward one location, as in Fig. 4.5(b). Next suppose Taylor is added to the array; then Wagner must be moved, as in Fig. 4.5(c). Last, suppose Davis is removed from the array. Then the five names Ford, Johnson, Smith, Taylor and Wagner must each be moved upward one location, as in Fig. 4.5(d). Clearly such movement of data would be very expensive if thousands of names were in the array.



The following algorithm inserts a data element ITEM into the K^{th} position in a linear array LA with N elements. The first four steps create space in LA by moving downward one location each element from the Kth position on. We emphasize that these elements are moved in reverse order- i.e., first LA[N], then LA [N - 1], ..., and last LA[K]; otherwise, data might be erased. In more detail, we first set J: = N and then, using J as a counter, decrease J each time the loop is executed until J reaches K. The next step, Step 5, inserts ITEM into the array in the space just created. Before the exit from the algorithm, the number N of elements in LA is increased by 1 to account for the new element.

Algorithm 4.2: (Inserting into a Linear Array) INSERT (LA, N, K, ITEM)

Here LA is a linear array with N elements and K is a positive integer such that $K \leq N$. This algorithm inserts an element ITEM into the Kth position in LA.

1. [Initialize counter.] Set J: = N.
2. Repeat Steps 3 and 4 while $J \geq K$.
3. [Move J^{th} element downward.] Set LA [J + 1] = LA[J].
4. [Decrease counter.] Set J: = J-1.
- [End of Step 2 loop.]
5. [Insert element.] Set LA[K]: = ITEM.
6. [Reset N.] Set N: = N + 1
7. Exit.

The following algorithm deletes the Kth element from a linear array LA and assigns it to a variable ITEM.

Algorithm 4.3: (Deleting from a Linear Array) DELETE (LA, N, K, ITEM)

Here LA is a linear array with N elements and K is a positive integer such that $K \leq N$. This algorithm deletes the Kth element from LA.

1. Set ITEM: = LA[K].
2. Repeat for J = K to N - 1:
 [Move J + 1st element upward.] Set LA[J]: = LA [J + 1].
 [End of loop.]
3. [Reset the number N of elements in LA.] Set N: = N - 1.
4. Exit.

4.6 Multi-Dimensional Array

Array is where each element is represented by more than one subscript is called multi-dimensional array.

Two-Dimensional Arrays

A two-dimensional $m \times n$ array A is a collection of $m.n$ data elements such that each element is specified by a pair of integers (such as J, K), called subscripts, with the property that

$$1 \leq J \leq m \quad \text{and} \quad 1 \leq K \leq n$$

The element of A with first subscript j and second subscript k will be denoted by

$$A_{J,K} \quad \text{or} \quad A[J,K]$$

Two-dimensional arrays are also called *matrix arrays*. There is a standard way of drawing a two-dimensional $m \times n$ array A where the elements of A form a rectangular array with m rows and n columns and where the element A [J, K] appears in row J and column K. A row is a *horizontal list of elements*, and a column is a *vertical list of elements*.

		Columns			
		1	2	3	4
Rows	1	A[1, 1]	A[1, 2]	A[1, 3]	A[1, 4]
	2	A[2, 1]	A[2, 2]	A[2, 3]	A[2, 4]
	3	A[3, 1]	A[3, 2]	A[3, 3]	A[3, 4]

Fig. 4.9 Two-Dimensional 3 × 4 Array A

Example 4.12

Suppose each student in a class of 25 students is given 4 tests. Assuming the students are numbered from 1 to 25, the test scores can be assigned to a 25 x 4 matrix array SCORE as pictured in Fig. 4.10. Thus SCORE [K, L] contains the Kth student's score on the Lth test. In particular, the second row of the array,

$$\text{SCORE}[2, 1], \text{SCORE}[2, 2], \text{SCORE}[2, 3], \text{SCORE}[2, 4]$$

contains the *four test scores of the second student*.

Student	Test 1	Test 2	Test 3	Test 4
1	84	73	88	81
2	95	100	88	96
3	72	66	77	72
⋮	⋮	⋮	⋮	⋮
25	78	82	70	85

Fig. 4.10 Array SCORE

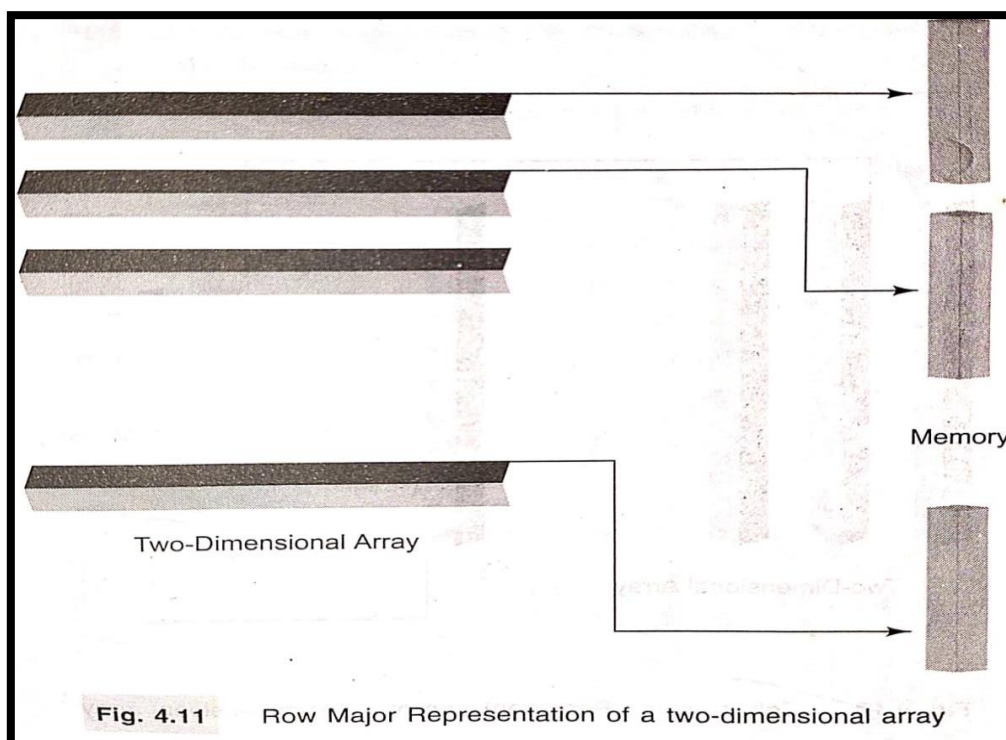
Suppose **A** is a two-dimensional **m x n** array. The first dimension of **A** contains the index set 1, 2, ..., **m** with *lower bound 1* and *upper bound m*; The second dimension of **A** contains the index set 1, 2, ..., **n** with *lower bound 1* and *upper bound n*. The **length** of a dimension is the number of integers in its index set. The pair of lengths **m x n** (read "m by n") is called the **size of the array**. The length of a given dimension (i.e., the number of integers in its index set) can be obtained from the formula

$$\text{Length} = \text{upper bound} - \text{lower bound} + 1$$

Generally speaking, unless otherwise stated, we will always assume that our arrays are regular, that is, that the lower bound of any dimension of an array is equal to 1.

Storage Representation

Row-major Representation



A two-dimensional array can be assumed as a single column with many rows and mapped sequentially. Such representation is called **Row-Major Representation**. Shown in figure 4.11

We can calculate the address of the element of the m^{th} row and the n^{th} column in a two-dimensional array using the formula shown below:

$$\text{addr}(a[m, n]) = (\text{total number of rows present before the } m^{\text{th}} \text{ row} \times \text{size of a row}) + (\text{total number of elements present before the } n^{\text{th}} \text{ element in the } m^{\text{th}} \text{ row} \times \text{size of element})$$

In the above equation:

The total number of rows present before m^{th} row = $(m - \text{lb1})$ and

lb1 is a first dimensional lower bound.

Size of a row = total number of elements present in a row $\times$ size of an element.

Total number of elements in a row calculated using $(\text{ub2} - \text{lb2} + 1)$ and

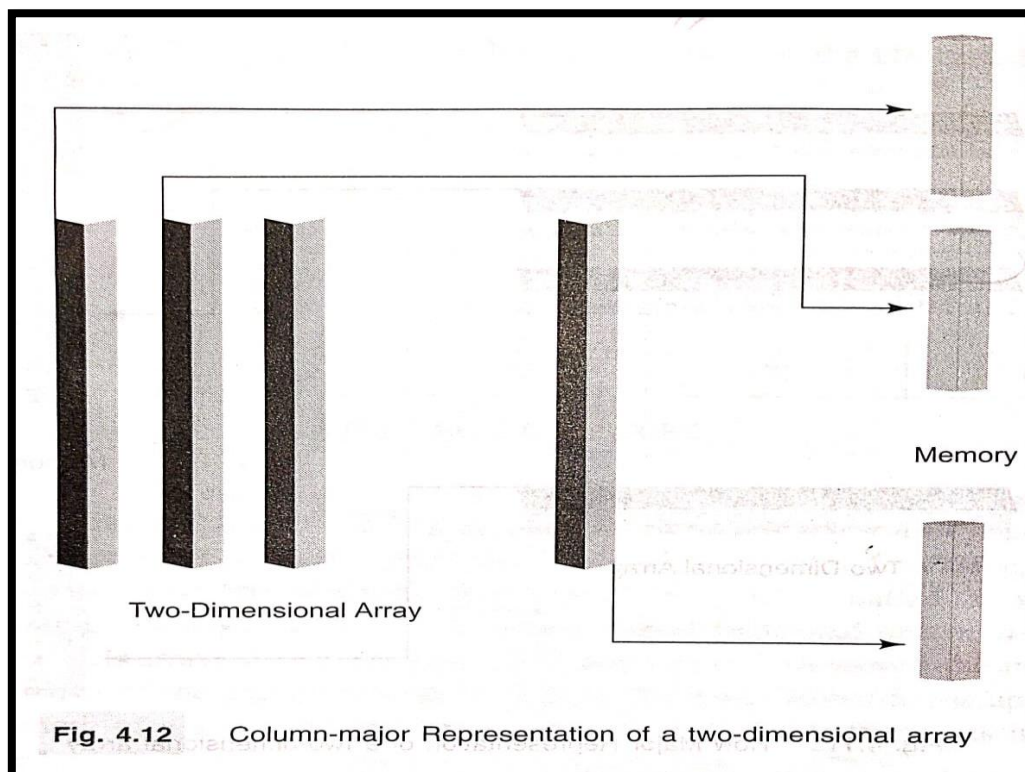
ub2 and lb2 are the second dimensional upper and lower bounds.

Now that we have a clear idea about all the variables present the above equation can be written

$$\text{addr}(a[m, n]) = ((m - \text{lb1}) \times (\text{ub2} - \text{lb2} + 1) \times \text{size}) + ((n - \text{lb2}) \times \text{size})$$

Column-major Representation

A two-dimensional array represented as one single row of multiple columns and map it sequentially. Such a representation is called a **column-major representation**. Shown in Fig. 4.12



We can calculate the address of the element of the m^{th} row and the n^{th} column as follows:

$$\text{addr}(a[m, n]) = (\text{Total number of columns present before } n \text{ column} \times \text{column size}) + (\text{Total number of elements present before the } m \text{ element of the } n \text{ column} \times \text{each elements size})$$

Columns which are placed before n^{th} column = $(n - lb_2)$ where lb_2 is a second dimensional lower bound.

Column size = Total number of elements present in a column $\times$ Element size
Number of elements in a column = $(ub_1 - lb_1 + 1)$.

Here ub_1 is first dimensional upper bound and lb_1 first dimensional lower bound.

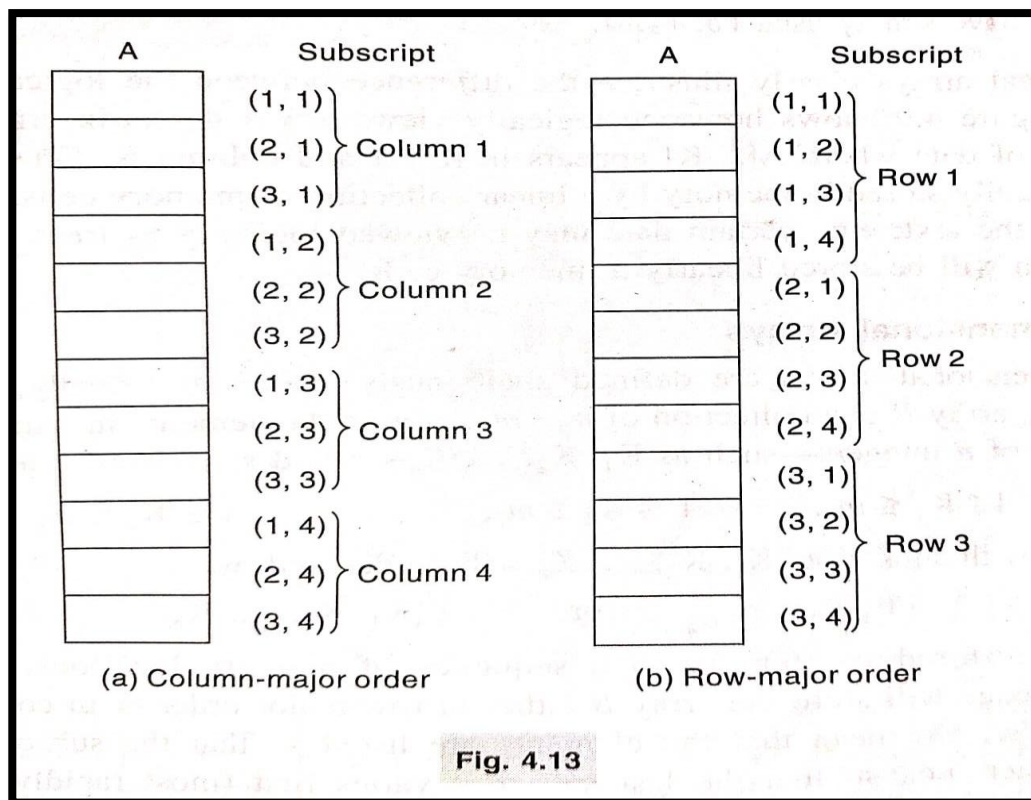
Therefore:

$$\text{addr}(a[m, n]) = ((n - lb_2) \times (ub_1 - lb_1 + 1) \times \text{size}) + ((m - lb_1) \times \text{size})$$

Representation of Two-Dimensional Arrays in Memory

Let A be a two-dimensional $m \times n$ array. Although A is pictured as a rectangular array of elements with m rows and n columns, the array will be represented in memory by a block of $m.n$ sequential memory locations. Specifically, the programming language will store the array A either (1) column by column, is called column major order, or (2) row by row, in row-major order.

Figure 4.13 shows these two ways when A is a two-dimensional 3×4 array.



For a linear array LA, the computer does not keep track of the address LOC(LA[K]) of every element LA[K] of LA, but does keep track of Base(LA), the address of the first element of LA. The computer uses the formula

$$\text{LOC(LA[K])} = \text{Base (LA)} + w(\text{K}-1) \text{ or}$$

$$\text{LOC(LA[K])} = \text{Base (LA)} + w(\text{K}-\text{LB})$$

to find the address of LA[K] in time independent of K.

(Here w is the number of words per memory cell for the array LA, and 1 is the **lower bound** of the index set of LA.)

A similar situation also holds for any two-dimensional $m \times n$ array A. That is, the computer keeps track of Base (A)-the address of the first element A[1, 1] of A-and computes the address LOC(A[J, K]) of A[J, K] using the formula

$$\text{(Column-major order) } \text{LOC(A[J, K])} = \text{Base (A)} + w[\text{M(K-1)} + (\text{J}-1)]$$

or the formula

$$\text{(Row-major order) } \text{LOC(A[J, K])} = \text{Base (A)} + w[\text{N(J-1)} + (\text{K}-1)]$$

Again, w denotes the number of words per memory location for the array A. Note that the formulas are linear in J and K, and that one can find the address LOC (A [J, K]) in time independent of J and K.

Example 4.13

Consider the 25 x 4 matrix array SCORE in Example 4.12. Suppose Base(SCORE) = 200 and there are w = 4 words per memory cell. Furthermore, suppose the programming language stores two-dimensional arrays using row-major order. Then the address of SCORE[12, 3], the third test of the twelfth student, follows:

$$\text{LOC(A[J, K])} = \text{Base (A)} + w[\text{N(J-1)} + (\text{K}-1)]$$

$$\text{LOC(SCORE[12, 3])} = 200 + 4[4(12-1) + (3-1)]$$

$$= 200 + 4[46]$$

$$= 384$$

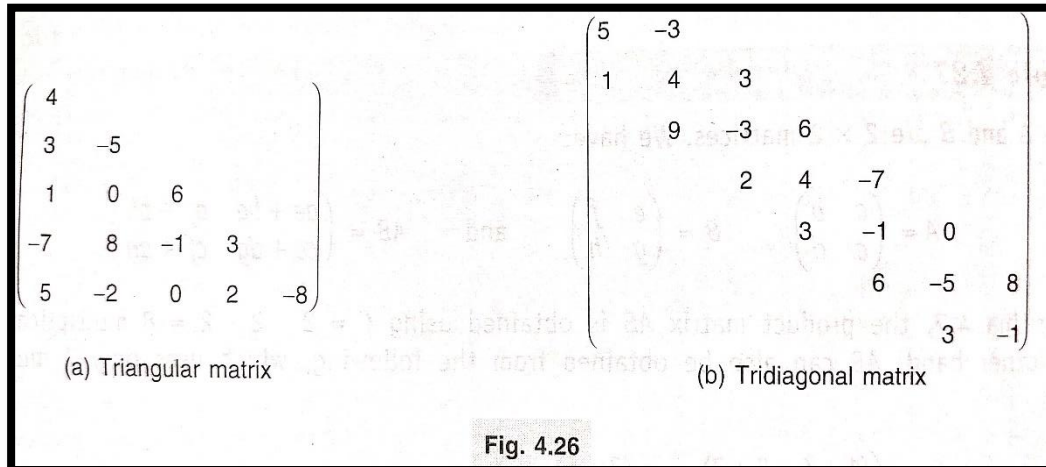
4.6 Sparse Matrices

Matrices with a relatively high proportion of zero entries are called **sparse matrices**. Two general types of n-square sparse matrices, which occur in various applications, are pictured in Fig. 4.26. (It is sometimes customary to omit blocks of zeros in a matrix as in Fig. 4.26.)

The matrix, where all entries above the main diagonal are zero or, equivalently, where nonzero entries can only occur on or below the main diagonal, is called a (Lower) **triangular matrix**.

The matrix, where all entries below the main diagonal are zero or, equivalently, where nonzero entries can only occur on or above the main diagonal, is called a **(Upper) triangular matrix**.

The matrix, where nonzero entries can only occur on the diagonal or on elements immediately above or below the diagonal, is called a **tridiagonal matrix**.



Unit-1

Chapter-5

Sorting

5.1 Introduction

Sorting is the process of Arranging the records in some logical order.

1. Alphabetical Order
2. Numerical Order

Alphabetical Order according to some NAME key, Numerical Order according to some NUMBER key, such as social security number or account number.

Some of the Sorting Techniques are as follows:

- Selection Sort
- Bubble Sort
- Quick Sort
- Insertion Sort
- Merge Sort

Selection Sort

Selection sort is an effective and efficient sort algorithm based on comparison operations. It adds one element in each iteration. You need to select the smallest element in the array and move it to the beginning of the array by swapping with the front element. You can also accomplish this by selecting the most potent element and positioning it at the back end. In each iteration, selection sort selects an element and places it in the appropriate position.

In selection sort, the first smallest element is selected from the unsorted array and placed at the first position. After that second smallest element is selected and placed in the second position. The process continues until the array is entirely sorted. Suppose an array A with n elements $A[1], A[2], \dots, A[N]$ is in memory. The selection sort algorithm for sorting A works as follows. First find the smallest element in the list and put it in the first position. Then find the second smallest element in the list and put it in the second position. And so on. More precisely:

Pass 1. Find the location LOC of the smallest in the list of N elements
A[1], A[2], ..., A[N], and then interchange A[LOC] and A[1]. Then: A[1] is sorted.

Pass 2. Find the location LOC of the smallest in the sublist of N – 1 elements
A[2], A[3], ..., A[N], and then interchange A[LOC] and A[2]. Then:
A[1], A[2] is sorted, since $A[1] \leq A[2]$.

Pass 3. Find the location LOC of the smallest in the sublist of N – 2 elements
A[3], A[4], ..., A[N], and then interchange A[LOC] and A[3]. Then:
A[1], A[2], ..., A[3] is sorted, since $A[2] \leq A[3]$.

...

...

Pass N – 1. Find the location LOC of the smaller of the elements A[N – 1], A[N], and then
interchange A[LOC] and A[N – 1]. Then:
A[1], A[2], ..., A[N] is sorted, since $A[N - 1] \leq A[N]$.

Thus A is sorted after N – 1 passes.

QB: Trace the following number using selection sort technique: 77, 33, 44, 11, 88, 22, 66, 55

Suppose an array A contains 8 elements as follows:

77, 33, 44, 11, 88, 22, 66, 55

Applying the selection sort algorithm to A yields the data in Fig. 9.5. Observe that LOC gives the location of the smallest among A[K], A[K+1],, A[N] during Pass K. The circled elements indicate the elements which are to be interchanged.

Pass	A[1]	A[2]	A[3]	A[4]	A[5]	A[6]	A[7]	A[8]
K = 1, LOC = 4	77	33	44	11	88	22	66	55
K = 2, LOC = 6	11	33	44	77	88	22	66	55
K = 3, LOC = 6	11	22	44	77	88	33	66	55
K = 4, LOC = 6	11	22	33	77	88	44	66	55
K = 5, LOC = 8	11	22	33	44	88	77	66	55
K = 6, LOC = 7	11	22	33	44	55	77	66	88
K = 7, LOC = 7	11	22	33	44	55	66	77	88
Sorted:	11	22	33	44	55	66	77	88

Fig. 9.5 Selection Sort for $n = 8$ Items

There remains only the problem of finding, during the Kth pass, the location LOC of the smallest among the elements A[K], A[K + 1], ..., A[N]. This may be accomplished by using a variable MIN to hold the current smallest value while scanning the subarray from A[K] to

$A[N]$. Specifically, first set $MIN := A[K]$ and $LOC := K$, and then traverse the list, comparing MIN with each other element $A[J]$ as follows:

- a) If $MIN \leq A[J]$, then simply move to the next element.
- b) If $MIN > A[J]$, then update MIN and LOC by setting $MIN := A[J]$ and $LOC := J$.

After comparing MIN with the last element $A[N]$, MIN will contain the smallest among the elements $A[K]$, $A[K + 1]$, $A[N]$ and LOC will contain its location. The above process will be stated separately as a procedure.

Procedure 9.2: $MIN(A, K, N, LOC)$

An array A is in memory. This procedure finds the location LOC of the smallest element among $A[K]$, $A[K + 1]$, ..., $A[N]$.

1. Set $MIN := A[K]$ and $LOC := K$. [Initializes pointers.]
2. Repeat for $J = K + 1, K + 2, \dots, N$:
 If $MIN > A[J]$, then: Set $MIN := A[J]$ and $LOC := J$ and $LOC := J$.
 [End of loop.]
3. Return.

Algorithm 9.3: (Selection Sort) $SELECTION(A, N)$

This algorithm sorts the array A with N elements.

1. Repeat Steps 2 and 3 for $K = 1, 2, \dots, N - 1$:
2. Call $MIN(A, K, N, LOC)$.
3. [Interchange $A[K]$ and $A[LOC]$.]
 Set $TEMP := A[K]$, $A[K] := A[LOC]$ and $A[LOC] := TEMP$.
 [End of Step 1 loop.]
4. Exit.

Bubble Sort

Bubble sort is a simple sorting algorithm. This sorting algorithm is comparison-based algorithm in which each pair of adjacent elements is compared and the elements are swapped if they are not in order.

At the end of this first pass, the largest number, 85, has moved to the last position. However, the rest of the numbers are not sorted, even though some of them have changed their positions. For the remainder of the passes, we show only the interchanges.

Pass 2. (27), (33), 51, 66, 23, 13, 57, 85
 27, 33, 51, (23), (66), 13, 57, 85
 27, 33, 51, 23, (13), (66), 57, 85
 27, 33, 51, 23, 13, (57), (66), 85
 At the end of Pass 2, the second largest number, 66, has moved its way down to the next-to-last position.

Pass 3. 27, 33, (23), (51), 13, 57, 66, 85
 27, 33, 23, (13), (51), 57, 66, 85

Pass 4. 27, (23), (33), 13, 51, 57, 66, 85
 27, 23, (13), (33), 51, 57, 66, 85

Pass 5. (23), (27), 13, 33, 51, 57, 66, 85
 23, (13), (27), 33, 51, 57, 66, 85

Pass 6. (13), (23), 27, 33, 51, 57, 66, 85

Pass 6 actually has two comparisons, A_1 with A_2 and A_2 and A_3 . The second comparison does not involve an interchange.

Pass 7. Finally, A_1 is compared with A_2 . Since $13 < 23$, no interchange takes place.

Since the list has 8 elements; it is sorted after the seventh pass. (Observe that in this example, the list was actually sorted after the sixth pass. This condition is discussed at the end of the section.)

The bubble sort algorithm as follows

Observe that there is an inner loop which is controlled by the variable PTR, and the loop is contained in an outer loop which is controlled by an index K. Also observe that PTR is used as a subscript but K is not used as a subscript, but rather as a counter.

Algorithm 4.4: (Bubble Sort) BUBBLE(DATA, N)

Here DATA is an array with N elements. This algorithm sorts the elements in DATA.

1. Repeat Steps 2 and 3 for $K = 1$ to $N - 1$.
2. Set $PTR := 1$. [Initializes pass pointer PTR.]
3. Repeat while $PTR \leq N - K$: [Executes pass.]
 - (a) If $DATA[PTR] > DATA[PTR + 1]$, then:
Interchange $DATA[PTR]$ and $DATA[PTR + 1]$.
[End of If structure.]
 - (b) Set $PTR := PTR + 1$.

[End of inner loop.]
[End of Step 1 outer loop.]
4. Exit.

Insertion Sort

Suppose an array A with n elements $A[1], A[2], \dots, A[N]$ is in memory. The insertion sort algorithm scans A from $A[1]$ to $A[N]$, inserting each element $A[K]$ into its proper position in the previously sorted subarray $A[1], A[2], \dots, A[K - 1]$. That is:

- Pass 1. $A[1]$ by itself is trivially sorted.
- Pass 2. $A[2]$ is inserted either before or after $A[1]$ so that: $A[1], A[2]$ is sorted.
- Pass 3. $A[3]$ is inserted into its proper place in $A[1], A[2]$, that is, before $A[1]$, between $A[1]$ and $A[2]$, or after $A[2]$, so that: $A[1], A[2], A[3]$ is sorted.
- Pass 4. $A[4]$ is inserted into its proper place in $A[1], A[2], A[3]$ so that:
- $A[1], A[2], A[3], A[4]$ is sorted.

- Pass N. $A[N]$ is inserted into its proper place in $A[1], A[2], \dots, A[N - 1]$ so that:
- $A[1], A[2], \dots, A[N]$ is sorted.

This sorting algorithm is frequently used when n is small. For example, this algorithm is very popular with bridge players when they are first sorting their cards. There remains only the problem of deciding how to insert $A[K]$ in its proper place in the sorted subarray $A[1], A[2], \dots, A[K - 1]$. This can be accomplished by comparing $A[K]$ with $A[K - 1]$, comparing $A[K]$ with $A[K - 2]$, comparing $A[K]$ with $A[K - 3]$, and so on, until first meeting an element $A[J]$ such that $A[J] \leq A[K]$. Then each of the elements $A[K-1], A[K-2], \dots, A[J + 1]$ is moved forward one location, and $A[K]$ is then inserted in the $J + 1^{\text{st}}$ position in the array.

The algorithm is simplified if there always is an element $A[J]$ such that $A[J] \leq A[K]$; otherwise we must constantly check to see if we are comparing $A[K]$ with $A[1]$. This condition can be accomplished by introducing a sentinel element $A[0] = \infty$ (or a very small number).

Example 9.4

Suppose an array A contains 8 elements as follows:

77, 33, 44, 11, 88, 22, 66, 55

Figure 9.4 illustrates the insertion sort algorithm. The circled element indicates the $A[K]$ in each pass of the algorithm, and the arrow indicates the proper place for inserting $A[K]$.

Pass	A[0]	A[1]	A[2]	A[3]	A[4]	A[5]	A[6]	A[7]	A[8]
K = 1:	$-\infty$	(77)	33	44	11	88	22	66	55
K = 2:	$-\infty$	← 77	(33)	44	11	88	22	66	55
K = 3:	$-\infty$	33	← 77	(44)	11	88	22	66	55
K = 4:	$-\infty$	← 33	44	77	(11)	88	22	66	55
K = 5:	$-\infty$	11	33	44	77	(88)	22	66	55
K = 6:	$-\infty$	11	← 33	44	77	88	(22)	66	55
K = 7:	$-\infty$	11	22	33	44	← 77	88	(66)	55
K = 8:	$-\infty$	11	22	33	44	← 66	77	88	(55)
Sorted:	$-\infty$	11	22	33	44	55	66	77	88

Fig. 9.4 Insertion Sort for $n = 8$ Items

The insertion sort algorithm as follows

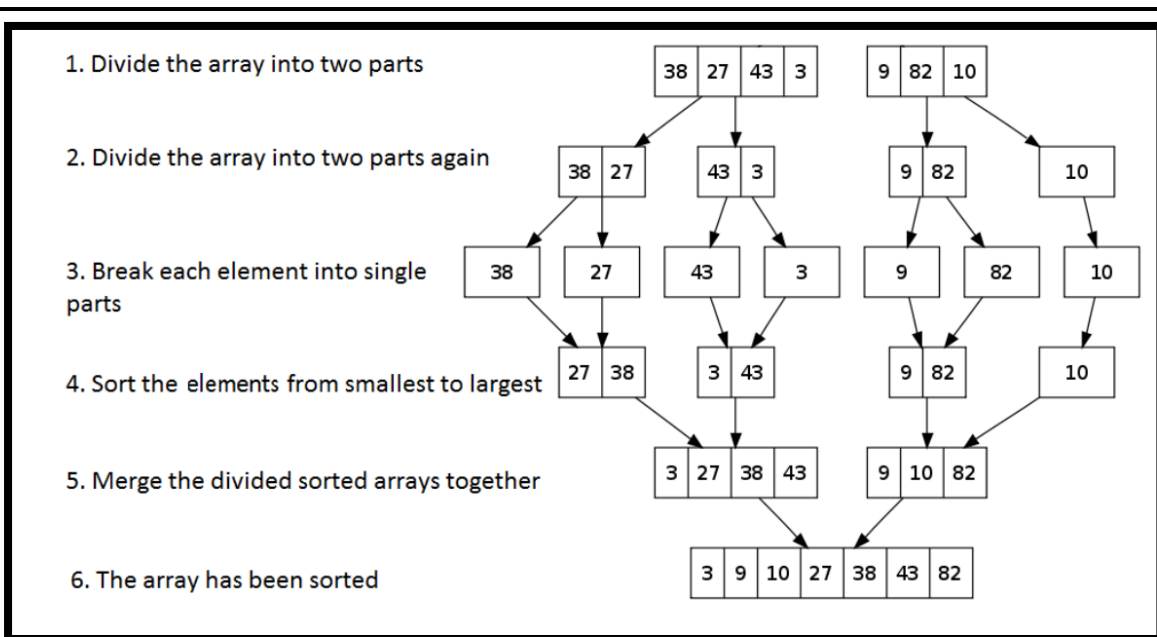
Algorithm 9.1: (Insertion Sort) INSERTION(A, N).

This algorithm sorts the array A with N elements.

1. Set $A[0] := -\infty$. [Initializes sentinel element.]
2. Repeat Steps 3 to 5 for $K = 2, 3, \dots, N$:
3. Set $TEMP := A[K]$ and $PTR := K - 1$.
4. Repeat while $TEMP < A[PTR]$:
 - (a) Set $A[PTR + 1] := A[PTR]$. [Moves element forward.]
 - (b) Set $PTR := PTR - 1$.
 [End of loop.]
5. Set $A[PTR + 1] := TEMP$. [Inserts element in proper place.]
[End of Step 2 loop.]
6. Return.

Merge Sort

It divides the given list into two equal halves, calls itself for the two halves and then merges the two sorted halves. The sub-lists are divided again and again into halves until the list cannot be divided further. Then we combine the pair of one element lists into two-element lists, sorting them in the process. The sorted two-element pairs is merged into the four-element lists, and so on until we get the sorted list. Suppose an array A with n elements $A[1], A[2], \dots, A[N]$ is in memory. The merge-sort algorithm which sorts A will first be described by means of a specific example.

**Example 9.7**

Suppose the array A contains 14 elements as follows:

66, 33, 40, 22, 55, 88, 60, 11, 80, 20, 50, 44, 77, 30

Each pass of the merge-sort algorithm will start at the beginning of the array A and merge pairs of sorted subarrays as follows:

Pass 1. Merge each pair of elements to obtain the following list of sorted pairs:

33, 66 22, 40 55, 88 11, 60 20, 80 44, 50 30, 70

Pass 2. Merge each pair of pairs to obtain the following list of sorted quadruplets:

22, 33, 40, 66 11, 55, 60, 88 20, 44, 50, 80 30, 77

Pass 3. Merge each pair of sorted quadruplets to obtain the following two sorted subarrays:

11, 22, 33, 40, 55, 60, 66, 88 20, 30, 44, 50, 77, 80

Pass 4. Merge the two sorted subarrays to obtain the single sorted array

11, 20, 22, 30, 33, 40, 44, 50, 55, 60, 66, 77, 80, 88

The original array A is now sorted.

The above merge-sort algorithm for sorting an array A has the following important property. After Pass K, the array A will be partitioned into sorted subarrays where each subarray, except possibly the last, will contain exactly $L = 2^K$ elements. Hence the algorithm requires at most $\log n$ passes to sort an n -element array A. The above informal description of merge-sort will now be translated into a formal algorithm which will be divided into two parts. The first part will be a procedure MERGEPASS, which uses Procedure 9.5 to execute a single pass of the algorithm; and the second part will repeatedly apply MERGEPASS until A is sorted.

The MERGEPASS procedure applies to an n -element array A which consists of a sequence of sorted subarrays. Moreover, each subarray consists of L elements except that the

last subarray may have fewer than L elements. Dividing n by $2*L$, we obtain the quotient Q , which tells the number of pairs of L -element sorted subarrays; that is,

$$Q = \text{INT}(N/(2*L))$$

(We use $\text{INT}(X)$ to denote the integer value of X .) Setting $S = 2*L*Q$, we get the total number S of elements in the Q pairs of subarrays. Hence $R = N - S$ denotes the number of remaining elements. The procedure first merges the initial Q pairs of L -element subarrays. Then the procedure takes care of the case where there is an odd number of subarrays (when $R \leq L$) or where the last subarray has fewer than L elements.

The formal MergePass and the Merge-Sort algorithm as follows:

Procedure 9.6: MERGEPASS(A, N, L, B)

The N -element array A is composed of sorted subarrays where each subarray has L elements except possibly the last subarray, which may have fewer than L elements. The procedure merges the pairs of subarrays of A and assigns them to the array B .

1. Set $Q := \text{INT}(N/(2*L))$, $S := 2*L*Q$ and $R := N - S$.
2. [Use Procedure 9.5 to merge the Q pairs of subarrays.]
 Repeat for $J = 1, 2, \dots, Q$:
 - (a) Set $LB := 1 + (2*J - 2)*L$. [Finds lower bound of first array.]
 - (b) Call MERGE($A, L, LB, A, L, LB + L, B, LB$).
 [End of loop.]
3. [Only one subarray left?]
 If $R \leq L$, then:
 - Repeat for $J = 1, 2, \dots, R$:
 - Set $B(S + J) := A(S + J)$.
 [End of loop.]
 Else:
 - Call MERGE($A, L, S + 1, A, R, L + S + 1, B, S + 1$).
 [End of If structure.]
4. Return.

Algorithm 9.7: MERGESORT(A, N)

This algorithm sorts the N -element array A using an auxiliary array B .

1. Set $L := 1$. [Initializes the number of elements in the subarrays.]
2. Repeat Steps 3 to 5 while $L < N$:
3. Call MERGEPASS(A, N, L, B).
4. Call MERGEPASS($B, N, 2 * L, A$).
5. Set $L := 4 * L$.
 [End of Step 2 loop.]
6. Exit.

Quick Sort

Quicksort is an algorithm of the divide-and-conquer type. That is, the problem of sorting a set is reduced to the problem of sorting two smaller sets. We illustrate this "reduction step" by means of a specific example.

Suppose A is the following list of 12 numbers:

(44), 33, 11, 55, 77, 90, 40, 60, 99, 22, 88, (66)

The reduction step of the quicksort algorithm finds the final position of one of the numbers; in this illustration, we use the first number, 44. This is accomplished as follows. Beginning with the last number, 66, scan the list from right to left, comparing each number with 44 and stopping at the first number less than 44. The number is 22. Interchange 44 and 22 to obtain the list

(22), 33, 11, 55, 77, 90, 40, 60, 99, (44), 88, 66

(Observe that the numbers 88 and 66 to the right of 44 are each greater than 44.) Beginning with 22, next scan the list in the opposite direction, from left to right, comparing each number with 44 and stopping at the first number greater than 44. The number is 55. Interchange 44 and 55 to obtain the list

22, 33, 11, (44), 77, 90, 40, 60, 99, (55), 88, 66

(Observe that the numbers 22, 33 and 11 to the left of 44 are each less than 44.) Beginning this time with 55, now scan the list in the original direction, from right to left, until meeting the first number less than 44. It is 40. Interchange 44 and 40 to obtain the list

22, 33, 11, (40), 77, 90, (44), 60, 99, 55, 88, 66

(Again, the numbers to the right of 44 are each greater than 44.) Beginning with 40, scan the list from left to right. The first number greater than 44 is 77. Interchange 44 and 77 to obtain the list

22, 33, 11, (40), 44, 90, (77), 60, 99, 55, 88, 66

(Again, the numbers to the left of 44 are each less than 44.) Beginning with 77, scan the list from right to left seeking a number less than 44. We do not meet such a number before meeting 44. This means all numbers, have been scanned and compared with 44. Furthermore, all numbers less than 44 now form the sublist of numbers to the left of 44, and all numbers greater than 44 now form the sublist of numbers to the right of 44, as shown below:

22, 33, 11, 40, (44), 90, 77, 60, 99, 55, 88, 66
 First sublist Second sublist

Thus 44 is correctly placed in its final position, and the task of sorting the original list A has now been reduced to the task of sorting each of the above sublists.

The above reduction step is repeated with each sublist containing 2 or more elements. Since we can process only one sublist at a time, we must be able to keep track of some sublists for future processing. This is accomplished by using two stacks, called LOWER and UPPER, to temporarily "hold" such sublists. That is, the addresses of the first and last elements of each sublist, called its boundary values, are pushed onto the stacks LOWER and UPPER, respectively; and the reduction step is applied to a sublist only after its boundary values are removed from the stacks. The following example illustrates the way the stacks LOWER and UPPER are used.

Example 6.8

Consider the above list A with $n = 12$ elements. The algorithm begins by pushing the boundary values 1 and 12 of A onto the stacks to yield

LOWER: 1 UPPER: 12

In order to apply the reduction step, the algorithm first removes the top values 1 and 12 from the stacks, leaving

LOWER: (empty) UPPER: (empty)

and then applies the reduction step to the corresponding list A[1], A[2], ..., A[12]. The reduction step, as executed above, finally places the first element, 44, in A[5]. Accordingly, the algorithm pushes the boundary values 1 and 4 of the first sublist and the boundary values 6 and 12 of the second sublist onto the stacks to yield

LOWER: 1, 6 UPPER: 4, 12

In order to apply the reduction step again, the algorithm removes the top values, 6 and 12, from the stacks, leaving

LOWER: 1 UPPER: 4

and then applies the reduction step to the corresponding sublist A[6], A[7], ..., A[12]. The reduction step changes this list as in Fig. 6.13. Observe that the second sublist has only one element. Accordingly, the algorithm pushes only the boundary values 6 and 10 of the first sublist onto the stacks to yield

LOWER: 1, 6 UPPER: 4, 10

And so on. The algorithm ends when the stacks do not contain any sublist to be processed by the reduction step.

A[6],	A[7],	A[8],	A[9],	A[10],	A[11],	A[12],
90,	77,	60,	99,	55,	88,	66
66,	77,	60,	99,	55,	88,	90
66,	77,	60,	90,	55,	88,	99
66,	77,	60,	88,	55,	90,	99
First sublist				Second sublist		

Fig. 6.13

The formal statement of our quicksort algorithm follows is divided into two parts. The first part gives a procedure, called QUICK, which executes the above reduction step of the algorithm, and the second part uses QUICK to sort the entire list.

Observe that Step 2(c)(iii) is unnecessary. It has been added to emphasize the symmetry between Step 2 and Step 3. The procedure does not assume the elements of A are distinct. Otherwise, the condition $LOC \neq RIGHT$ in Step 2(a) and the condition $LEFT \neq LOC$ in Step 3(a) could be omitted.

In second part of the algorithm LOWER and UPPER are stacks on which the boundary values of the sublists are stored. (As usual, we use $NULL = 0$.)

Procedure 6.7: QUICK(A, N, BEG, END, LOC)

Here A is an array with N elements. Parameters BEG and END contains the boundary values of the sublist of A to which this procedure applies. LOC keeps track of the position of the first element A[BEG] of the sublist during the procedure. The local variables LEFT and RIGHT will contain the boundary values of the list of elements that have not been scanned.

1. [Initialize] Set $LEFT := BEG$, $RIGHT := END$ and $LOC := BEG$
2. [Scan from right to left.]
 - (a) Repeat while $A[LOC] \leq A[RIGHT]$ and $LOC = RIGHT$
 $RIGHT := RIGHT - 1$
 [End of loop]
 - (b) If $LOC = RIGHT$, then: Return.
 - (c) If $A[LOC] > A[RIGHT]$, then:
 - (i) [Interchange $A[LOC]$, $A[LOC] := A[RIGHT]$].
 $TEMP := A[LOC]$, $A[LOC] := A[RIGHT]$, $A[RIGHT] := TEMP$.
 - (ii) Set $LOC := RIGHT$
 - (iii) Go to Step 3.
- [End of If Structure.]
3. [Scan from left to right]
 - (a) Repeat while $A[LEFT] \leq A[LOC]$ and $LEFT \neq LOC$:
 $LEFT := LEFT + 1$
 [End of loop.]
 - (b) If $LOC = LEFT$, then: Return.
 - (c) If $A[LEFT] > A[LOC]$, then
 - (i) [Interchange $A[LEFT]$ and $A[LOC]$.]
 $TEMP := A[LOC]$, $A[LOC] := A[LEFT]$, $A[LEFT] := TEMP$.

- (ii) Set LOC := LEFT.
- (iii) Go to Step 2.
- [End of If Structure.]

Algorithm 6.8: (Quicksort) This algorithm sorts an array A with N elements.

1. [Initialize.] TOP := NULL.
2. [Push boundary values of A onto stacks when A has 2 or more elements.]
If $N > 1$, then: TOP := TOP + 1, LOWER[1] := 1, UPPER[1] := N.
3. Repeat Steps 4 to 7 while TOP $\neq$ NULL.
4. [Pop sublist from stacks.]
Set BEG := LOWER[TOP], END := UPPER[TOP],
TOP := TOP - 1.
5. Call QUICK(A, N, BEG, END, LOC). [Procedure 6.7.]
6. [Push left sublist onto stacks when it has 2 or more elements.]
If BEG < LOC - 1, then:
 TOP := TOP + 1, LOWER[TOP] := BEG,
 UPPER[TOP] := LOC - 1.
[End of If structure.]
7. [Push right sublist onto stacks when it has 2 or more elements.]
If LOC + 1 < END, then:
 TOP := TOP + 1, LOWER[TOP] := LOC + 1,
 UPPER[TOP] := END.
[End of If structure.]
[End of Step 3 loop.]
8. Exit.